CLIENTDELIVERABLE

# Vehicle Selector Pro

## A Production-Grade Shopify App for Vehicle Fitment

Project Report / August 2026

**Live storefront demo:** https://vehicle-selector-pro.fly.dev/demo

**Live merchant admin:** https://vehicle-selector-pro.fly.dev/demo/admin

**Source repository:** https://github.com/ai-dev-2024/vehicle-selector-pro

A detailed engineering and delivery report — architecture, workflows, data model, security, quality, the live demo catalog, and the free-AI build provenance.

## Contents

## 1. Executive Summary

Vehicle Selector Pro is a complete Ruby on Rails application that lets Shopify merchants map vehicle attributes (Year, Make, Model, Trim, Engine) to their products and gives customers a dynamic, cascading fitment widget on the storefront. It ships as a production-grade Rails app with a normalized local data cache, Shopify Metafield sync, a Theme App Extension widget, App Proxy endpoints, GraphQL data integration, and asynchronous processing via ActiveJob and Sidekiq — engineered from the start for the Shopify App Store.

**Four pillars that make it production-grade:**

- **Normalized local cache.** Fitments live in structured ActiveRecord tables (48 YMMTE configurations, 204 verified fitments, 40 products, 8 brands in the demo), so filtering is instant and never pounds Shopify's API.
- **Shopify Metafield sync.** Fitment data is written to the custom.vehicle_fitment product metafield via the GraphQL metafieldsSet mutation, batched to stay inside API limits, so product pages render their own fitment badge.
- **Cascading storefront widget.** Year → Make → Model → Trim → Engine menus narrow the catalog behind an HMAC-signed App Proxy, returning only parts that fit the selected vehicle.
- **Async, resilient background jobs.** Webhooks and syncs run on Sidekiq + Redis with exponential backoff and per-topic policies, so a traffic spike never blocks the request path.

## 2. The Problem & The Solution

Selling aftermarket parts is uniquely hard online. A brake pad or a wiper blade is not universally compatible: it depends on the exact Year, Make, Model, Trim and Engine (YMMTE) of the customer's vehicle. Getting this wrong means wrong parts, angry customers, and expensive returns. Merchants who can't afford a commercial fitment database either stop listing the data, drown shoppers in dense compatibility tables, or hard-code rules into individual product pages where they quickly rot and cost hours to maintain.

**Why existing shortcuts fall short.** Static compatibility tables bury the answer; manual per-product notes don't scale; paid fitment feeds lock merchants into contracts. Vehicle Selector Pro hands merchants control over their own fitment data — entered once, synced everywhere — without ongoing per-part fees or vendor lock-in.

Vehicle Selector Pro turns fitment from a per-product chore into a single, structured, reusable dataset. Fitments are entered once, stored in a normalized multi-tenant cache, and pushed to Shopify metafields — so the storefront, the product pages, and the merchant admin all stay in sync without ongoing per-part fees or brittle scripts.

**Four pillars that make it production-grade:**

- **Normalized local cache.** Fitments live in structured ActiveRecord tables (48 YMMTE configurations, 204 verified fitments, 40 products, 8 brands in the demo), so filtering is instant and never pounds Shopify's API.
- **Shopify Metafield sync.** Fitment data is written to the custom.vehicle_fitment product metafield via the GraphQL metafieldsSet mutation, batched to stay inside API limits, so product pages render their own fitment badge.
- **Cascading storefront widget.** Year → Make → Model → Trim → Engine menus narrow the catalog behind an HMAC-signed App Proxy, returning only parts that fit the selected vehicle.
- **Async, resilient background jobs.** Webhooks and syncs run on Sidekiq + Redis with exponential backoff and per-topic policies, so a traffic spike never blocks the request path.

## 3. Feature Highlights

Every capability is implemented against the real Shopify platform, not mocked.

### 1. Cascading storefront widget

CASCADING DROPDOWNS OVER HMAC-SIGNED APP PROXY DATA — Year, Make, Model, Trim, Engine dropdowns driven by HMAC-SHA256-signed App Proxy queries. Each selection returns only the options that actually exist, progressively narrowing the catalog until the customer reaches parts that fit their vehicle.

### 2. Product fitment badges

INSTANT COMPATIBILITY CLARITY ON THE PRODUCT PAGE — Every product dynamically renders a Guaranteed Exact Fit / Does NOT Fit / Universal Fit badge, so a shopper knows compatibility at a glance instead of reading a table or guessing.

### 3. My Garage

SAVED VEHICLES THAT FOLLOW THE SHOPPER — Shoppers save multiple vehicles and switch between them across pages via browser-local storage — a retention feature that turns repeat-fitment shoppers into returning customers.

### 4. Merchant admin dashboard

MANAGE EVERY FITMENT FROM ONE PLACE — A Polaris-styled command center: fitment matrix, vehicle library, bulk CSV import, sync monitor, and widget settings — all read from the normalized local database for fast, debuggable queries instead of live Shopify round-trips.

### 5. Metafield sync

RELIABLE, OBSERVABLE WRITES TO SHOPIFY — Fitment JSON is written to custom.vehicle_fitment via GraphQL metafieldsSet in batches of 25, with a per-shop sync log and progress tracking so merchants see exactly what synced and what failed.

### 6. Webhook pipeline

SHOPIFY EVENTS PROCESSED ASYNCHRONOUSLY AND SECURELY — products/*, app/uninstalled and shop/update events are HMAC-verified, then dispatched to Sidekiq jobs that keep the cache and tenant state correct — including graceful cleanup on uninstall and full GDPR data-erasure handlers.

### 7. Multi-tenant isolation

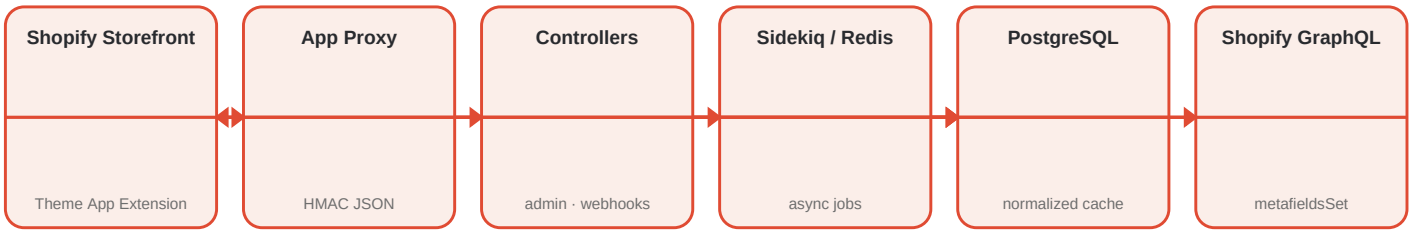
PER-SHOP ISOLATION AND ENCRYPTED TOKENS — Every query is scoped to the authenticated shop; OAuth tokens are encrypted at rest; and App Proxy / webhook traffic is verified with constant-time HMAC comparison.

### 8. Rate limiting & hardening

PROTECTED AGAINST ABUSE AND INJECTION — rack-attack throttles the App Proxy and public surface; payloads and signatures are validated; CSP frame-ancestors restrict embedding to myshopify.com.

## 4. Architecture

The system is a classic three-layer architecture. The Shopify storefront renders the Theme App Extension widget, which talks to the Rails application over signed App Proxy endpoints. The Rails app exposes admin controllers (Polaris UI), webhook endpoints, and App Proxy endpoints, all backed by a normalized PostgreSQL cache. Background Sidekiq jobs perform long-running work — metafield sync and webhook processing — exchanging data with Shopify's GraphQL Admin API. Redis provides the job queue and shared cache.




System diagram — storefront ↔ App Proxy ↔ cache, with Sidekiq driving Metafield writes through Shopify's GraphQL API.

## 5. Data Model & Multi-tenancy

The model is deliberately normalized for multi-tenant query speed. Each shop is a first-class tenant; all fitment, settings, and sync data are scoped to it.

### Shop

The tenant: Shopify domain, encrypted OAuth token, granted scopes, active flag, and GDPR timestamps.

### Vehicle

Normalized YMMTE row (year, make, model, trim, engine) plus drivetrain, body style, and transmission.

### VehicleProductFitment

The join that says 'this product fits this vehicle', with fitment type, notes, position, and a synced-to-metafield flag.

### AppSetting

Per-shop widget theming and behavior (labels, colors, which toggles are enabled).

### MetafieldSyncLog

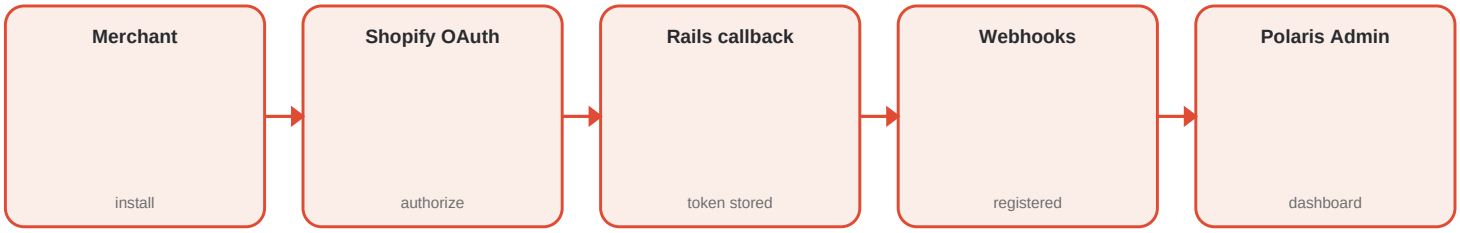
Audit trail of full and per-product syncs with status, counts, and error details.

## 6. Key Workflows

Two of the signature end-to-end flows, drawn step by step.

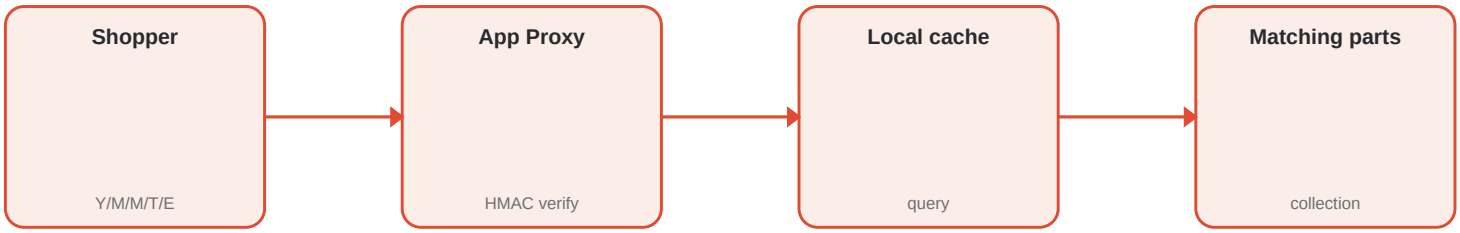
### 1. OAuth install

A merchant clicks the install link. shopify_app exchanges the code for an offline access token, stores the shop (encrypted), registers the webhooks, and admits the merchant to the Polaris dashboard — all through the standard shopify_app 22 flow.




### 2. Fitment selection

A shopper picks Year → Make → Model → Trim → Engine. Each request hits an App Proxy endpoint that verifies the HMAC signature, then queries the local cache to return only matching options.




### 3. Metafield sync

When fitments are saved, a BatchSyncJob builds a compact JSON payload per product and writes it via metafieldsSet in batches of 25, updating the sync log as it goes.

### 4. Webhook handling

Shopify posts product events. The controller verifies the HMAC, enqueues a Sidekiq job, and returns 200 immediately; the job updates cache rows or cleans up after deletes/uninstalls.

## 7. Technology Stack

| Layer | Stack |
|---|---|
| **Language / runtime** | Ruby 3.2 · Rails 7.1 |
| **Shopify integration** | shopify_app 22 · shopify_api 14 · GraphQL Admin API |
| **Database** | PostgreSQL (production) · SQLite3 (dev/test) |
| **Queuing** | Sidekiq 7 + Redis 5 (ActiveJob) |
| **UI styling** | Polaris View Components · ERB |
| **API & serialization** | Oj + ActiveModelSerializers |
| **Caching** | Cache-aware queries · stale-while-revalidate on Proxy responses |
| **Testing** | RSpec 7 · FactoryBot · WebMock · SimpleCov |
| **Code quality** | RuboCop (Rails/RSpec cops) · Bullet (N+1 detection) |
| **Security & hardening** | rack-attack · HMAC signature checks · encrypted tokens |
| **Hosting** | Fly.io (auto-deploy via GitHub Actions) |
| **CI/CD** | GitHub Actions — CI on push · Fly deploy · GitHub Pages report |

## 8. Security & Hardening

- OAuth 2.0 install with an offline access token, stored encrypted via Rails Active Record encryption.
- App Proxy requests verified with HMAC-SHA256 using constant-time comparison; synthetic params excluded.
- Webhook payloads verified against X-Shopify-Hmac-Sha256; unknown or inactive shops are dropped and logged.
- Every admin query scoped to the session's shop; shop identity is derived only from the verified session, never from request params.
- rack-attack throttling on the App Proxy and public endpoints.
- Content Security Policy frame-ancestors restricted to myshopify.com and admin.shopify.com.
- GDPR coverage: customers/data_request, customers/redact, shop/redact handlers plus an explicit public privacy policy.

## 9. Quality Engineering

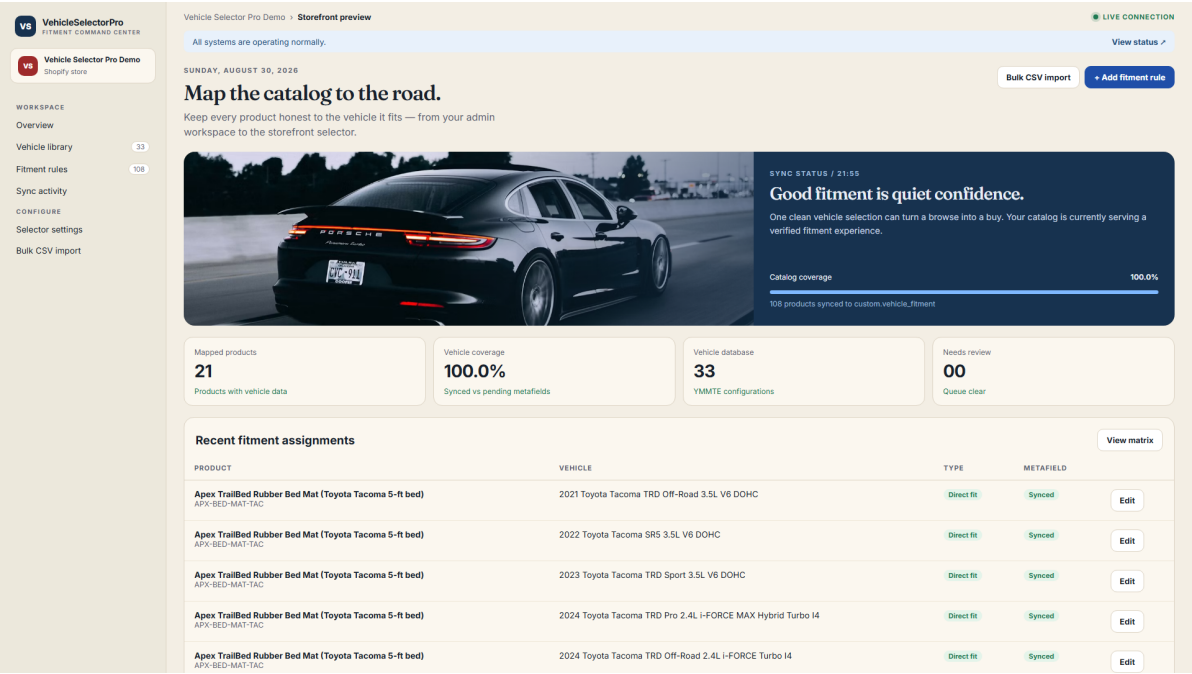
- 49 passing RSpec examples across models, services, request specs, and jobs.
- Webhook HMAC verification and App Proxy signature handling are exercised in request specs.
- Metafield sync payload generation is unit-tested (ownerId, namespace, key, JSON value shape).
- RuboCop with Rails & RSpec cops is clean across 90 files.
- CI on every push: full RSpec suite, RuboCop, and a Zeitwerk eager-load check — then auto-deploy to Fly.io on green.

## 10. Live Demo Catalog & Screenshots

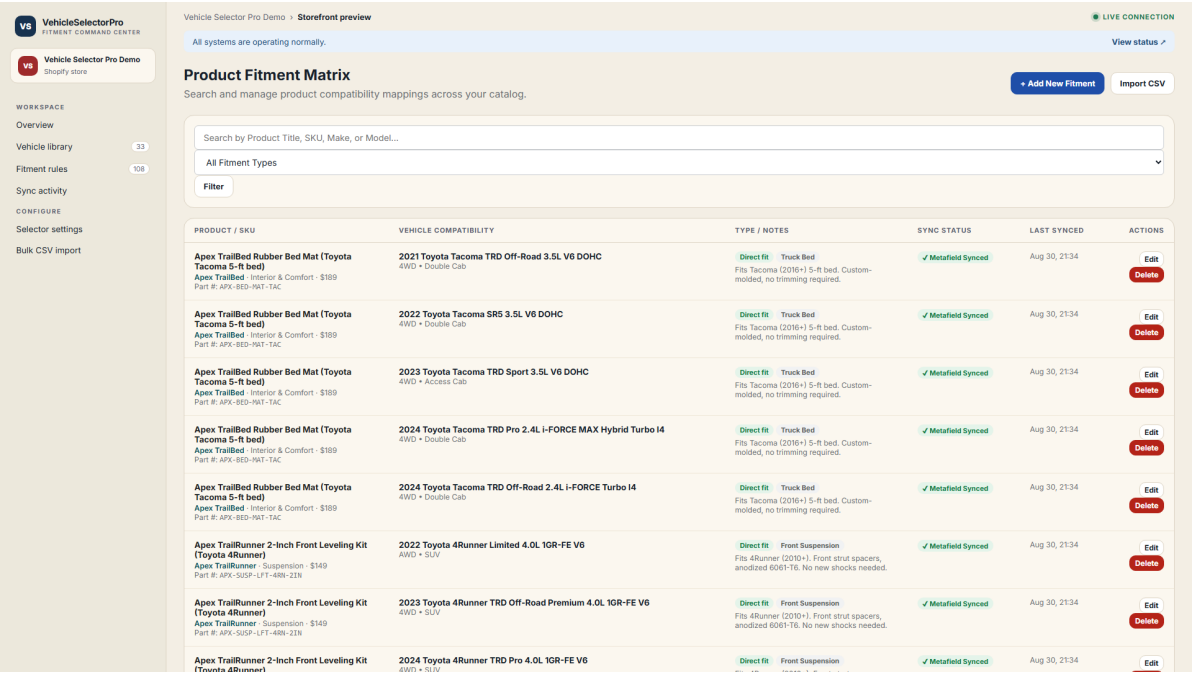
The live demo ships a real, browsable catalog — the numbers a merchant sees immediately, no setup required.

| | |
|---|---|
| **48** | **YMMTE configurations** |
| **204** | **verified fitments** |
| **40** | **mapped products** |
| **8** | **brands represented** |

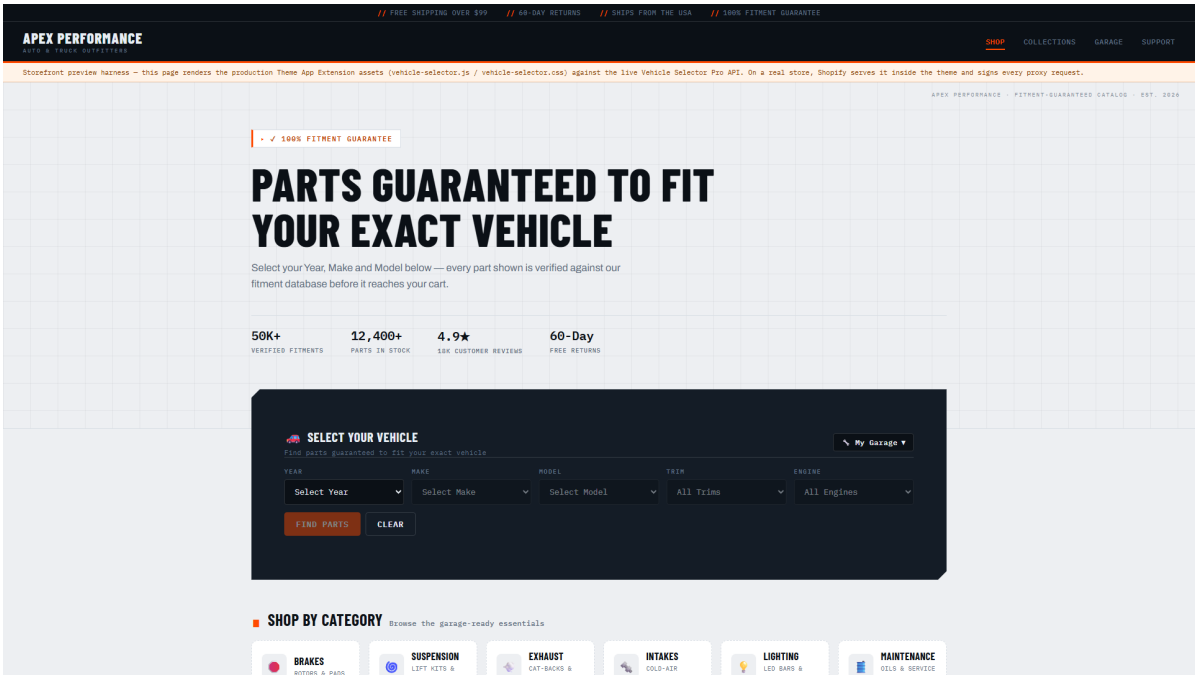
**Real screenshots of the running app** (storefront and merchant admin). Every image below is a full-width, high-resolution capture at native 1920×1080 — open the PDF to inspect it closely.



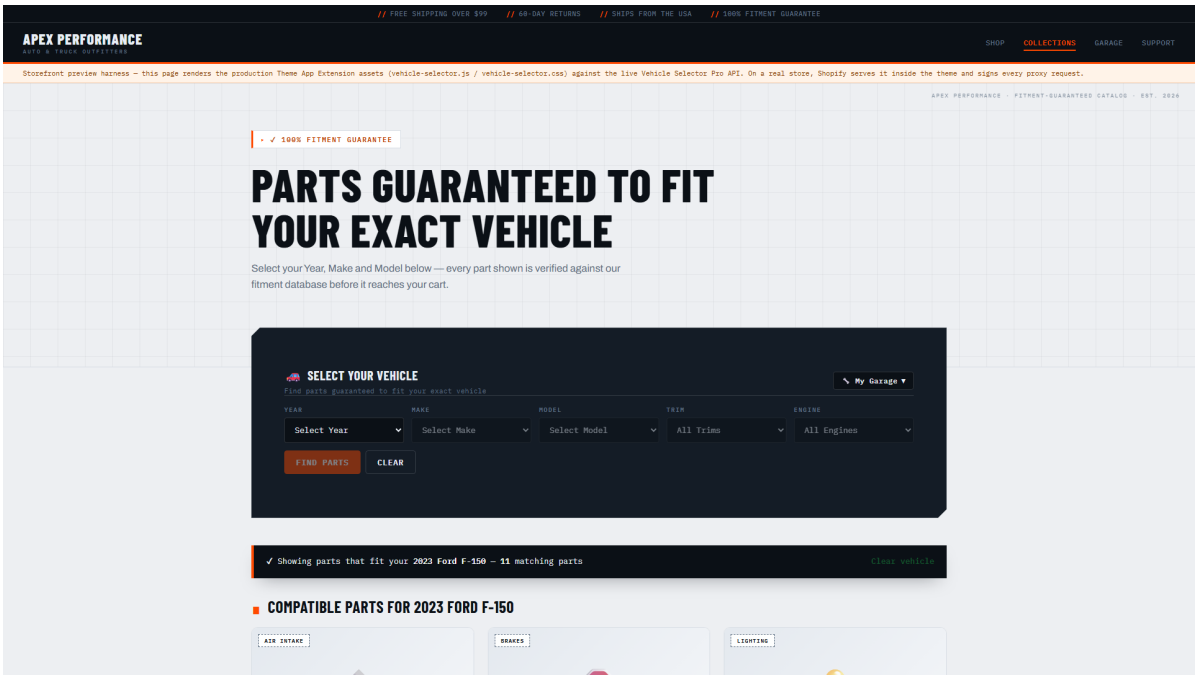

Admin dashboard — the merchant command center



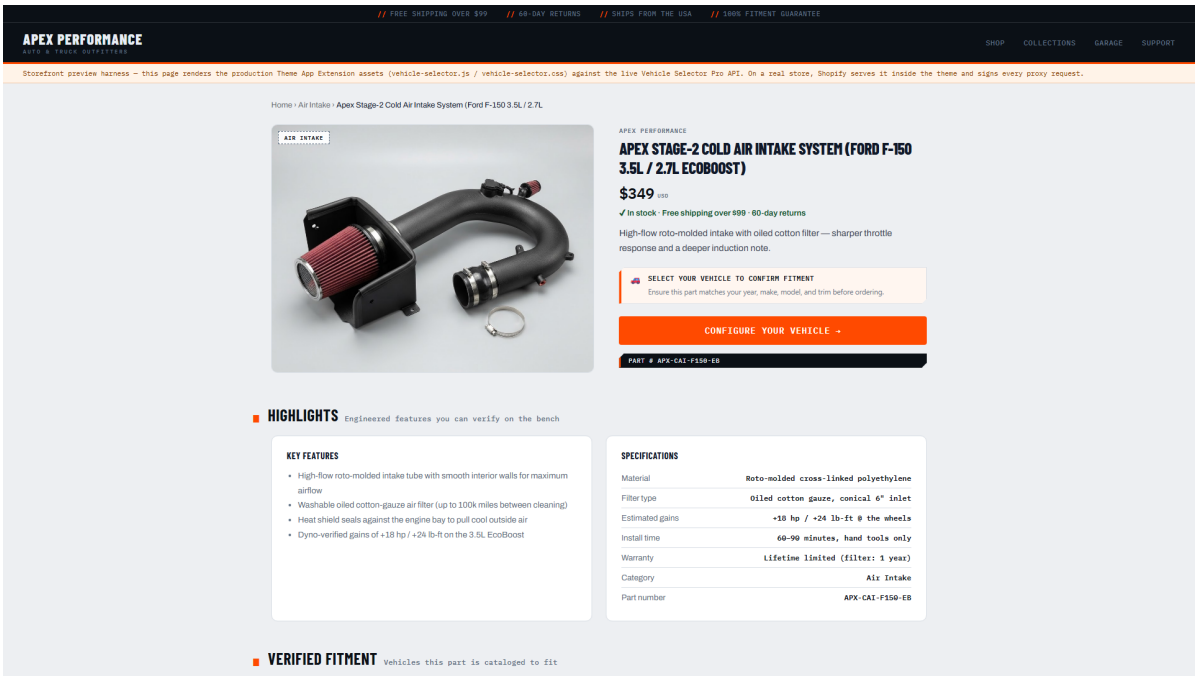

Fitment matrix — every product × vehicle rule in one view



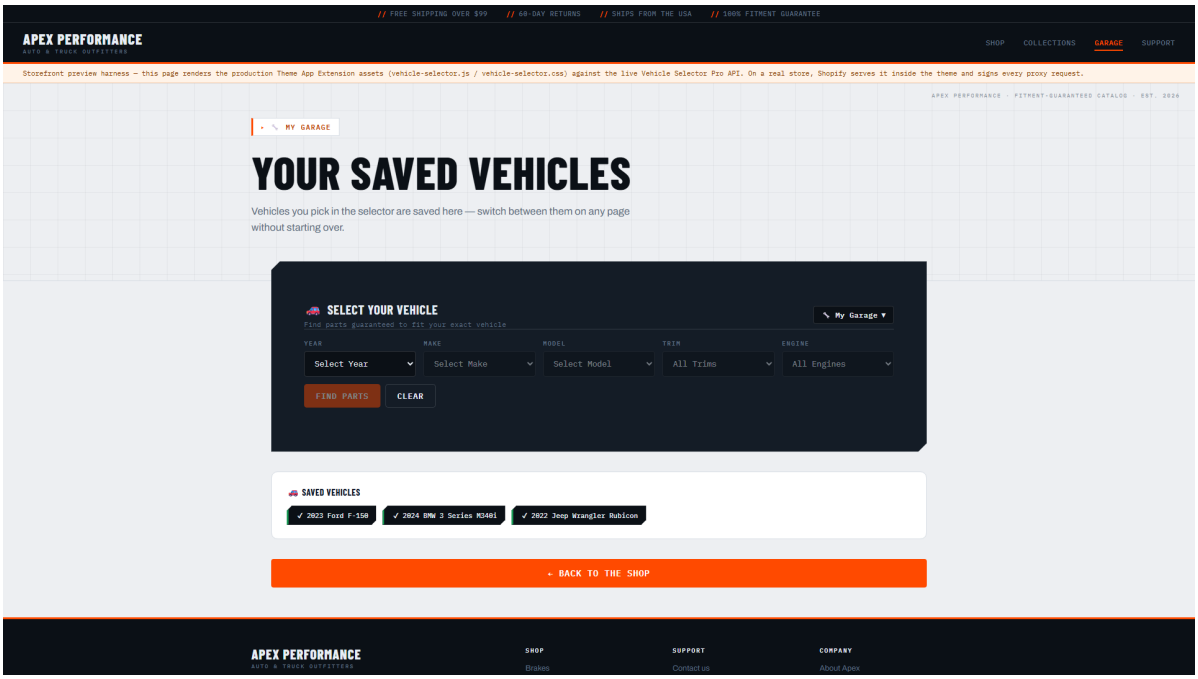

Storefront home — cascading vehicle selector widget



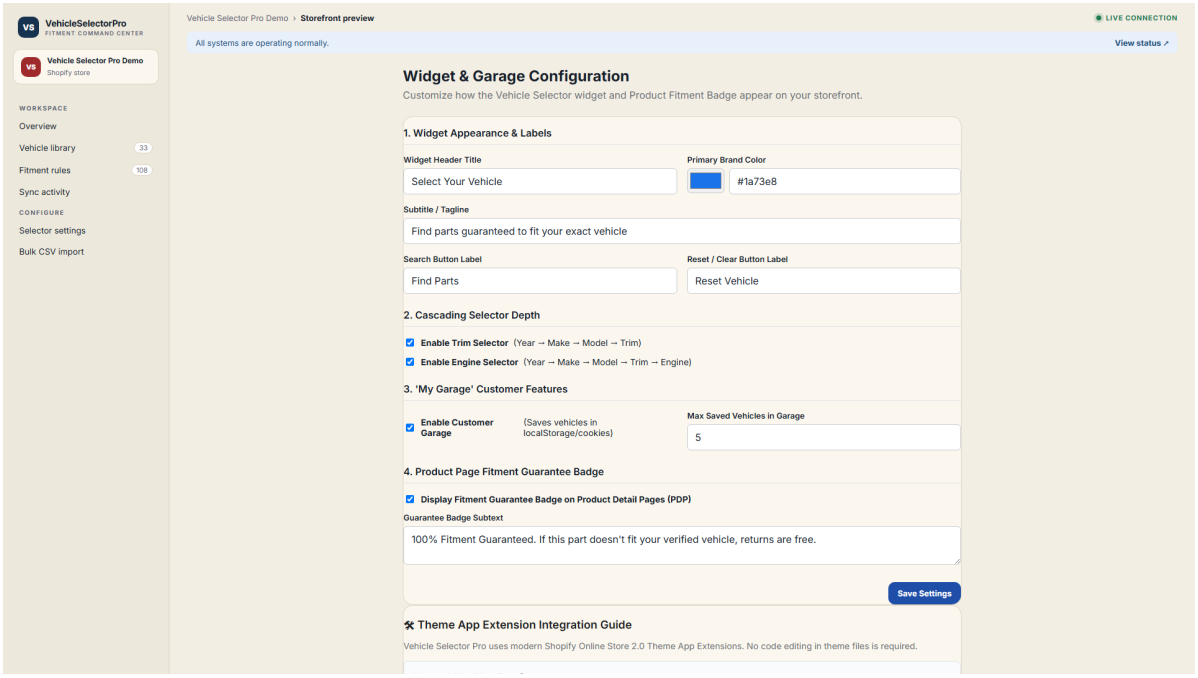

Filtered results — parts matching a 2023 Ford F-150



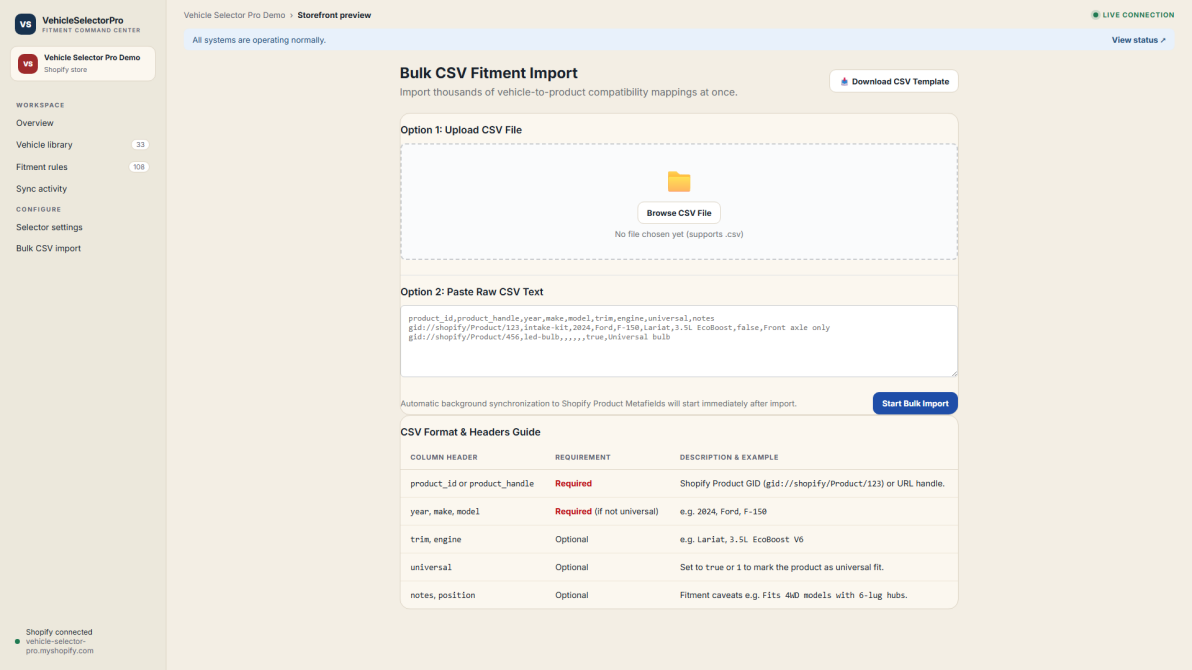

Product detail — spec sheet and live fitment badge



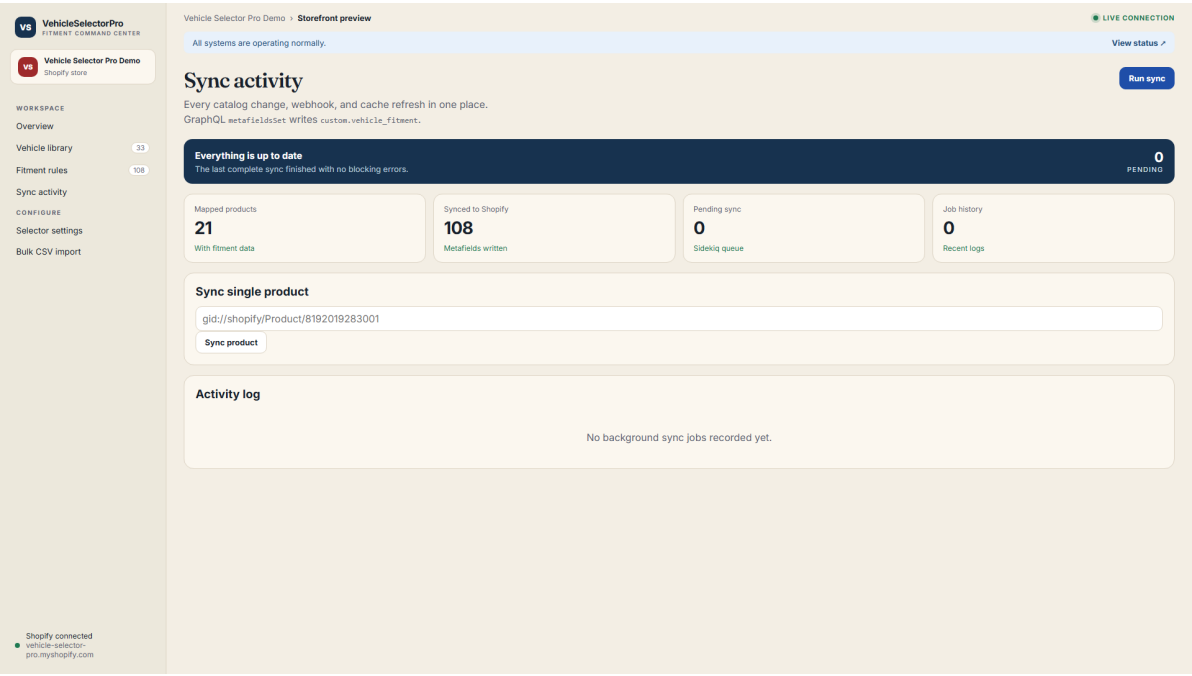

My Garage — saved vehicles shoppers can switch between



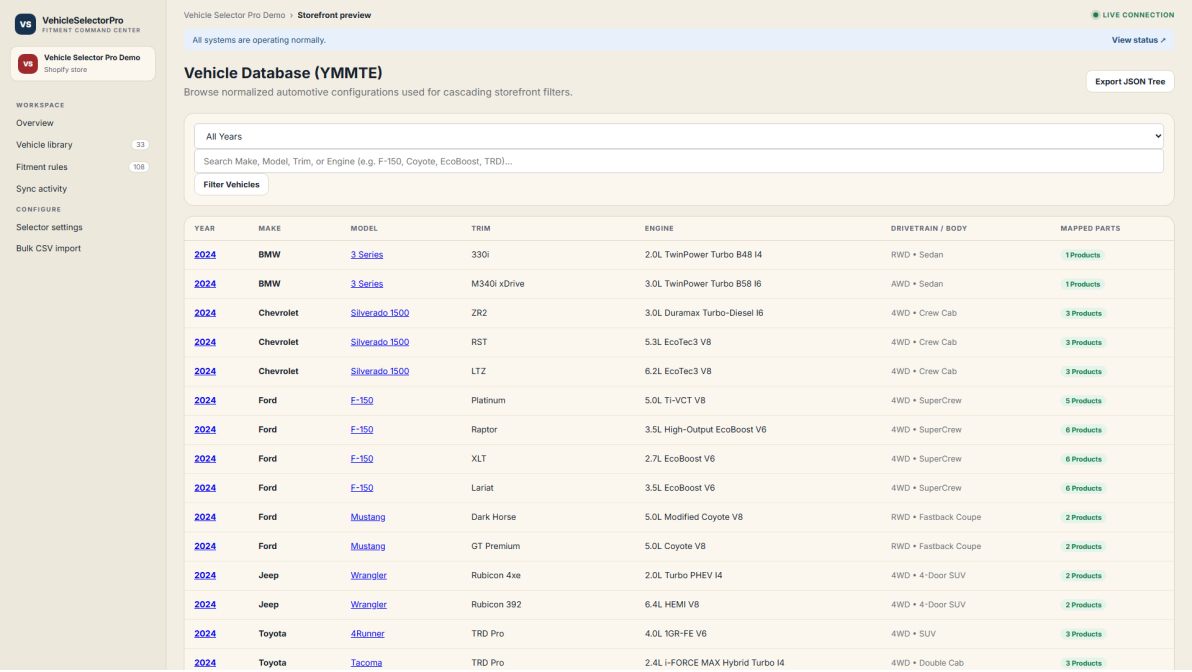

Widget settings — labels, colors, toggles




Bulk import — CSV fitment loader




Sync monitor — metafield sync progress




Vehicle library — the YMMTE catalogue

## Narrated demo videos

**Merchant command center:**
https://ai-dev-2024.github.io/vehicle-selector-pro/demo/vehicle-selector-pro-merchant.mp4

**Shopper experience:**
https://ai-dev-2024.github.io/vehicle-selector-pro/demo/vehicle-selector-pro-shopper.mp4

## 11. Deployment, Installation & Roadmap

The app runs on Fly.io with a private Redis for Sidekiq and a managed PostgreSQL database. Every push to main runs CI (full RSpec suite + RuboCop + Zeitwerk eager-load check) and, on success, auto-deploys to production. A health endpoint (/up) backs the load balancer, and structured logs feed standard observability tooling.

- CI: RSpec + RuboCop + Zeitwerk check on every push.
- Auto-deploy to Fly.io on green CI (drain-aware, zero-downtime Puma).
- Sidekiq on a private Redis instance; PostgreSQL for the normalized cache.
- Health endpoint (/up) for the load balancer and container orchestration.
- GitHub Pages hosts this client report automatically on every push.

### How to deploy & install on your own store

Anyone can take this codebase and run it on their own Shopify store. The complete path, in order:

#### 01 Create the app in Shopify Partners

In partners.shopify.com create an app, copy the API key & secret, and set the app URL plus the OAuth redirect URI (**https://<your-host>/auth/shopify/callback**). These become SHOPIFY_API_KEY and SHOPIFY_API_SECRET.

#### 02 Deploy the Rails app to your host

Fly.io (as here), Render, or any Rails host. Provide PostgreSQL, a Redis-backed queue, and the two Shopify env vars plus ActiveRecord encryption keys. It boots with bundle install + rails db:prepare.

#### 03 Grant the OAuth scopes

The app requests read_products/write_products (fitments + metafields), read_customers/write_customers, and read_product_listings so the storefront and syncing work.

#### 04 Register the App Proxy subpath

In the app's sales-channel settings add the App Proxy subpath /apps/vehicle-selector pointing at your Rails host; the widget queries those endpoints with an HMAC-signed querystring.

#### 05 Install on the store via OAuth

Open /login?shop=<store>.myshopify.com. shopify_app completes the OAuth flow, stores the shop, and auto-registers the webhooks (products/*, app/uninstalled, shop/update).

#### 06 One-time metafield definition

Create the custom.vehicle_fitment definition once in Admin → Settings → Custom Data (Product owner, type JSON, Public read). The app then writes every value via metafieldsSet automatically.

#### 07 Add the storefront widget

In the theme editor add the Vehicle Selector App Block and the Product Fitment Badge block from the Theme App Extension, then publish the theme.

### 08 Import fitments and go live

Bulk-import your fitment CSV (or assign rules in the admin), run a full metafield sync, and confirm the PDP badges and collection filters appear. Support and My Garage are enabled by default.

## Where it grows next — roadmap

- Shopify Billing API — recurring and one-time plans with managed merchant billing.
- Supersession & OE-number matching for a richer part-catalog engine.
- Fitment confidence scoring and a smarter universal/exact priority model.
- White-label storefront theming, translation support, and deeper widget controls.
- Merchant analytics: widget conversion lift, per-fitment demand, and drop-off reports.

Each item is production-feasible on the current architecture — built to grow without a rewrite.

## 12. Built With Free AI Tools on Minimal Hardware

No paid AI models, no commercial tooling — and the least compute you can imagine doing serious work with. Both are deliberate, and together they combine into the strongest case this project can make.

> Built with free AI tools. This project was designed, coded, and documented using no-cost AI assistance end to end — no paid AI models, APIs, or commercial tooling were used anywhere in its development. Everything visible here was produced with accessible, free tooling. That is deliberate: it demonstrates the ceiling of what free tooling already achieves, and it means the potential for further growth is massive — with paid, frontier AI models and paid tooling, this same foundation scales into an even richer, production-grade, store-ready solution with ease.

### The build machine: a 2018 ASUS ZenBook (netbook-class)

Everything in this report was produced locally on an everyday consumer notebook — no dedicated GPU, no developer-grade workstation — with details verified from the machine itself, not inferred. This is not a developer tool; it is the kind of laptop you would buy for browsing and documents, which is exactly the point: the result speaks to the headroom available once proper development hardware and tooling are allocated.

| | |
|---|---|
| **Device** | ASUS ZenBook UX433FA |
| **Class** | Netbook-class consumer laptop — not a developer machine |
| **CPU** | Intel Core i7-8565U @ 1.80 GHz (4 cores, ~15 W) |
| **Memory** | 16 GB RAM |
| **Graphics** | Intel UHD 620 — integrated only, no GPU |
| **Origin** | 2018-vintage consumer notebook |

Built at the ceiling of a consumer notebook. This entire project — the Rails application, the Shopify integration, the storefront, every diagram, this report, and even the two narrated walkthrough videos — was designed, coded, tested, and rendered on a single ASUS ZenBook UX433FA: a 2018-vintage, netbook-class consumer laptop. An Intel Core i7-8565U at 1.80 GHz (4 cores, ~15 W), 16 GB RAM, and only Intel UHD 620 integrated graphics. No GPU. No dedicated developer workstation. No cloud IDE. It is not a developer machine — it is the kind of everyday notebook you buy for browsing and documents — and every frame of video and every page of this report was produced on the CPU of that one machine. That constraint is deliberate and it is the strongest evidence of the opportunity: so far the ceiling was set by the hardware, not by the ambition. Put this same foundation on a proper development workstation with a GPU and paid, frontier AI models, and the architecture scales into a far richer, faster, store-ready solution. The headroom — and the case for investing in proper developer tools — is the opportunity.

## 13. Conclusion

The application is live, deployed, and healthy. Both the storefront shopping experience and the merchant command center are fully functional and can be explored immediately, and the two narrated walkthrough videos — merchant and shopper — demonstrate the complete flow from setup to purchase. Vehicle Selector Pro is ready for further development, App Store submission, or customization to a specific merchant catalog — and because it was built entirely with free AI tooling, the ceiling on that next stage is exceptionally high.

Vehicle Selector Pro · github.com/ai-dev-2024/vehicle-selector-pro · vehicle-selector-pro.fly.dev